Documento de lógica de negocio

Escenario 2: Pedidos con verificación geográfica de cobertura

1. Problema y solución

Este proyecto automatiza los pedidos de una tienda en línea. Antes de aceptar un pedido, el sistema verifica que el cliente tenga datos correctos, que su IP corresponda a Costa Rica y que su región tenga cobertura. Así se evita aceptar envíos que no se pueden realizar.

JSON Server funciona como una base de datos local sencilla: guarda los pedidos en un archivo y permite consultarlos o actualizarlos. n8n conecta los pasos de forma visual: recibe la solicitud, consulta los datos, aplica las reglas y actualiza el estado final.



Figura 1. Workflow final en n8n: rutas normales y rutas de error.

2. Recorrido de los datos

1. Webhook: recibe un POST con pedidoId e ip. 2. Buscar pedido: consulta el pedido en JSON Server. 3. Consultar IP: usa IP-API para conocer país y provincia. 4. Evaluar cobertura: aplica las clases y produce una decisión. 5. If: envía el pedido aceptado o rechazado al PATCH correspondiente.

3. Reglas de negocio aplicadas

Las reglas se implementan con dos clases de JavaScript dentro del nodo Evaluar cobertura. La clase Pedido valida que exista un cliente válido y un total numérico, finito y mayor que cero. También identifica si el total llega a ₡50 000 para optar por envío gratis.

La clase ZonaCobertura limpia tildes y mayúsculas de las provincias. El pedido se rechaza si la ubicación no puede verificarse, si no pertenece a Costa Rica (CR) o si la región no está reconocida. Zonas principales: San José, Heredia, Alajuela y Cartago: ₡2 500. Zonas extendidas: Guanacaste, Puntarenas y Limón: ₡4 500. El envío gratis se aplica solamente si la zona fue reconocida.

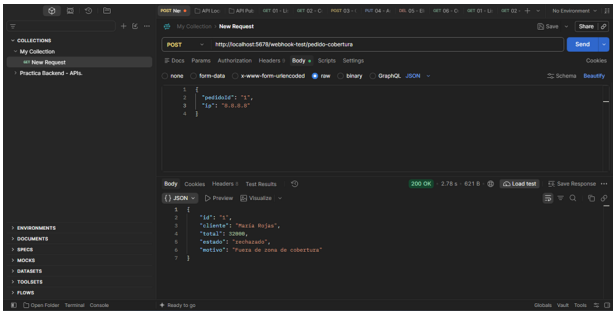


Figura 2. Prueba de datos incompletos: el pedido se rechaza.



Figura 3. Prueba de pedido inexistente con respuesta controlada.



Figura 4. Prueba fuera de cobertura: se rechaza el pedido.



Figura 5. Prueba exitosa: pedido aceptado y costo de envío ₡0.

4. Conexión entre POO y n8n

El nodo Code contiene una versión compatible de las clases Pedido y ZonaCobertura. n8n entrega los datos obtenidos en los nodos anteriores; el código crea los objetos, llama a evaluar(pedido, ubicacion) y devuelve estado, motivo y costoEnvio. El nodo If utiliza el estado para escoger la ruta correcta.

5. Evidencia de funcionamiento

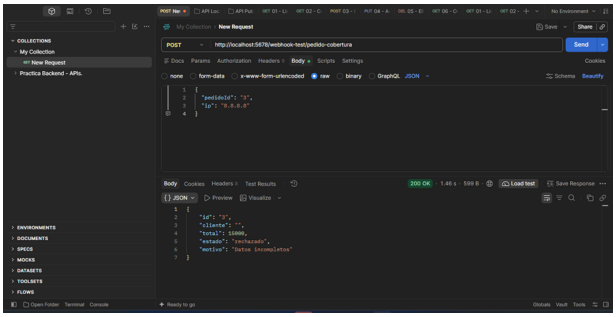


Figura 6. El nodo If lleva la ejecución hacia Rechazar pedido.

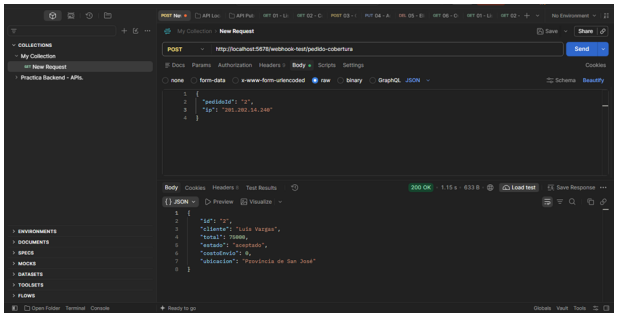


Figura 7. El nodo If lleva la ejecución hacia Aceptar pedido.

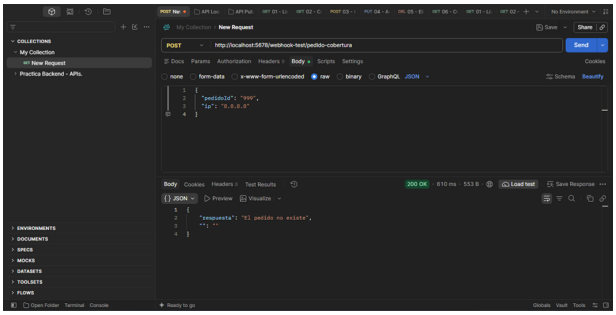


Figura 8. Vista del workflow con la ruta de error al consultar IP.

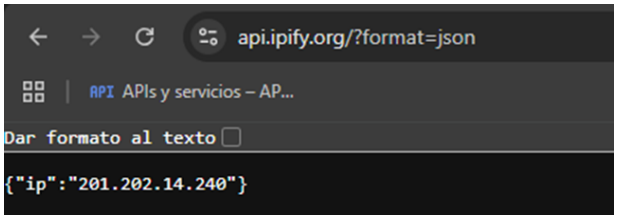


Figura 9. Consulta de IP pública utilizada para la prueba de aceptación.

6. Decisiones técnicas

JSON Server: se eligió porque permite simular una API local sin crear una base de datos real. Es adecuado para probar la integración y guardar el cambio de estado del pedido.

PATCH: se usa para actualizar únicamente los campos necesarios, por ejemplo estado, motivo o costoEnvio, sin reemplazar todo el pedido.

Manejo de errores: si el pedido no existe, el flujo responde de forma clara. Si IP-API falla técnicamente, la salida de error va a un nodo que genera un rechazo controlado y luego actualiza el pedido. Esto evita que el proceso termine sin una explicación.

Conclusión: el workflow combina automatización visual y POO para tomar decisiones consistentes. Cada pedido queda aceptado o rechazado con un motivo entendible y, cuando corresponde, con el costo de envío calculado.